

Distributed Deep Learning Inference at Edge Devices

FYP Project Proposal - Group 44

160170B	K.M.B.G.B Gunarathne
160477X	M.V.R.Perera
160628M	T.M.C.P Thennakoon

Table Of Contents

Table Of Contents	1
1. Introduction	2
1.1 Problem Statement	4
1.2. Research Objectives	5
2. Literature Review	6
2.1 Background	6
2.1.1 Background of DNN	6
2.1.2 Fully Connected Neural Network	7
2.1.3 Model Parallelism	8
2.2 Related Work	8
3. Methodology	11
3.1 Model Parallelism Based Partitioning	11
3.2 Communication and Task sharing	12
3.3 Research Timeline	14
4. Conclusion/Summary	14
References	15

1. Introduction

Deep neural networks are becoming an important tool in modern computing applications. Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) are heavily researched domains and are widely used in large scale data processing applications due to their higher classification accuracy compared to other methods[13]. In this context there are specialized deep neural networks introduced for various tasks. And also a lot of data and computation power is required to train/use these models.

Edge computing and IoT applications have emerged with the requirement of collecting and processing large sets of data locally within devices, thus, ensuring privacy and low latency. However, these edge devices are resource constrained and have less computational power, memory and energy compared to modern computers and servers. These edge devices may collect large amounts of sensory data where it needs to be processed and results computed in real-time for mission critical applications. Deploying Deep/Convolutional Neural Networks (DNNs & CNNs) applications on such mission critical resource constrained devices is a challenging task due to discussed conditions above. DNN/CNN has two phases which are called training and inference. Training of a neural network requires a large set of data and huge computational effort. Once the model is trained and ready it can be fed previously untrained input data and will return results with identified or classified patterns in the input data. This phase is called inference. Even Though it does not require huge computational power compared to training, executing inference tasks in resource constrained edge devices is still a challenging task [1].

Performing inference tasks of deep learning applications on edge devices ensures privacy of the input data and prevents latency and availability issues compared to a cloud solution. As most edge devices are resource constrained (eg : memory , computational) they cannot store and execute an entire Deep Neural Network(DNN). According to paper [12], One possible solution is to distribute the DNN across multiple edge devices. Layers in a neural network need to be carefully partitioned to balance the amount of computation

and memory usage on each resource constrained edge device and to reduce the communication among devices.

In this research we are focusing on the execution of the DNN inference part in resource constrained edge devices like Raspberry Pi boards. We intend to research and propose a novel approach to partition and distributively execute inference calculation on multiple edge devices which are connected in a local network. Furthermore, we plan to investigate how to efficiently reduce the communication demands among the devices.

1.1 Problem Statement

IoT devices have been facing the issue of low resources (e.g. computational capacity, memory, energy) since the beginning of the IoT era [10]. Thus, IoT applications that need to use computationally intensive technologies like augmented reality, video processing, cognitive assistance and machine vision are unable to get the best performance and real-time experience with these constraints.

Performing inference tasks of deep neural networks on IoT edge devices requires considerable computational power and memory space which are usually limited on IoT devices. Relatively high-end IoT boards such as Raspberry Pis also have limited resources eg: 1GB - 2GB RAM , 1GHz - 1.5GHz processor etc. (Depends on the versions.) Therefore, storing and execution of a large complete Deep Neural Network in a single device is an infeasible task. Even a medium scale DNN may present intolerable delays in executing inference calculations in these devices. The most common solution to this problem is cloud-based computation. This however introduces availability, privacy and latency issues. Local computing ensures the privacy of sensitive data and results in low latency.

When there are a large number of layers and also when there are a large number of neurons in one layer of a fully connected neural network or when there are computationally intensive convolutional layers, the resources of a Raspberry Pi device will either be not enough or it will take an unacceptable amount of time to compute the result.

Deep Learning Software Framework	MobileNet v1 (ms) Benchmark value	MobileNet V2 (ms) Benchmark value	Inception model that we tested (s)
TensorFlow Lite	271.5	379.6	24
TensorFlow	480.3	654.0	32

Table 1. A comparison of image classification time taken on a Raspberry Pi device

As shown in Table 1 above, Tensorflow optimised models like MobileNet v1 and v2 have acceptable low time consumption to complete execution, but we have tested a custom model (Inception model) an object detection library on our Raspberry Pi 3 B V1 setup, and it took an unacceptable amount of time compared to the Tensorflow given benchmark values [11].

We identified a lack of conducted research in this area to provide model partitioning and distributed execution in resource constrained IoT devices (etc. raspberry pi, mobile devices..) considering computational , memory and communication demands. So our problem is to find a more efficient way to run large Deep Neural Networks and improve the overall performance of distributed Deep Learning Inference on low resource IoT edge devices.

1.2. Research Objectives

The final intended outcome of this research is to propose a novel approach to build communication and memory aware partitioning and distribution schemes for fully connected neural network layers of deep neural networks among suitable resource constrained edge devices. Furthermore, a proof of concept prototype for distributed DNN/CNN execution is planned to be implemented.

- a. Discover model parallelism based partitioning scheme for fully connected neural networks.
- b. Propose a task distribution and task sharing mechanism among suitable resource constrained edge devices. (Eg: Raspberry Pi, mobile devices)
- c. Implement a proof of concept DNN/CNN partitioning and distributed processing application using C++.
- d. Compare performance of the developed system with existing systems.

2. Literature Review

In this section we first present some background information on neural networks. Then we proceed to discuss related work on distributed deep learning inference, challenges faced and the issues which are not yet addressed in the area.

2.1 Background

2.1.1 Background of DNN

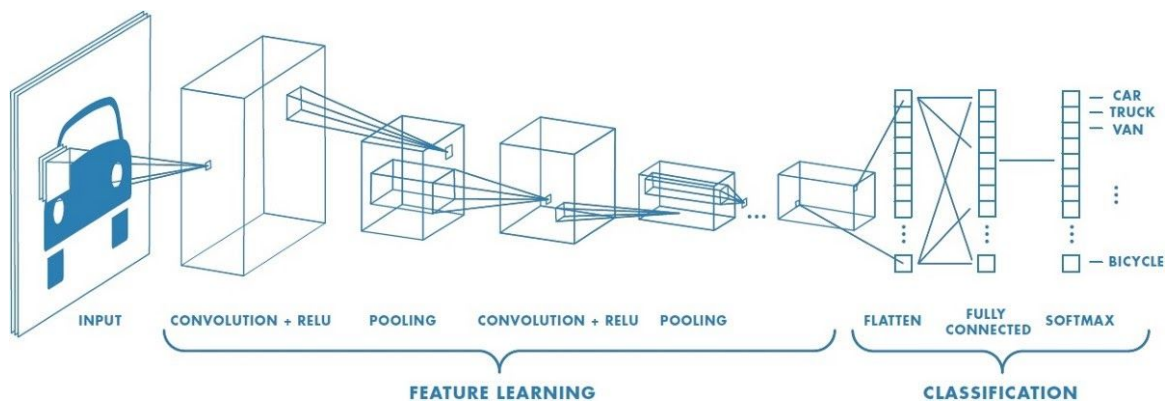


Figure 2.CNN architecture

The Figure 2 shows an overview of an Image classification Deep Neural Network with convolutional layers and fully connected layers. Convolutional layers are used to extract features (ie: edges, lines, curves, etc) in the given input. The fully connected network is used to classify the images, based on recognized features of the input data. Within these layers heavy computational tasks are executed while propagating through the network.

Within these convolutional layers it applies multiple pre-learned filters to their input to produce a set of feature maps which extract some features from the input. There can be many filters between each convolution layer which increase the computational and memory demand. Pooling layers are used to shrink the size of input feature maps.

2.1.2 Fully Connected Neural Network

Fully connected neural networks are usually used for recognizing and classification tasks. Therefore, convolution layers followed by fully connected layers are used in image recognition and classification. Fully connected layers consist of a number of neurons which have a pre learned bias value and activation function. These neurons are connected with all previous layer's neurons and the next layer's neurons where these connections have pre-learned weights.

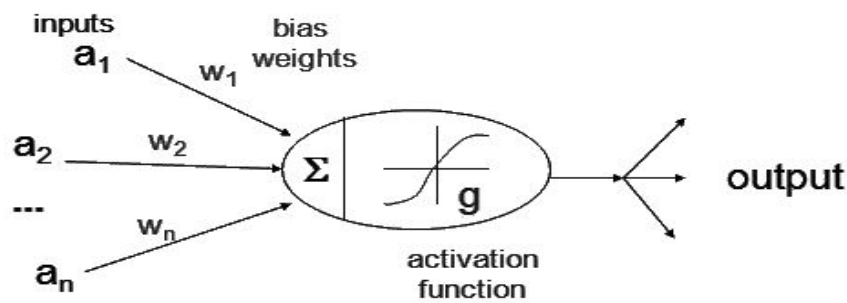


Figure 3. Activate Function process inside a neuron

The calculation inside each neurons can be described as follows

$$f\left(b + \sum_{i=1}^n a_i w_i\right)$$

Where

b = bias

a = input to neuron

w = weights

n = the number of inputs from the incoming layer

i = a counter from 0 to n

f = an Activation function

Therefore each calculation requires all input values of neurons for all connected non-zero weights. Therefore one possible solution will be to partition each calculation of neurons by sharing all input data and weight data to all devices. However, it will be not memory efficient as well as communication efficient.

2.1.3 Model Parallelism

Model parallelism is used to partition the neural network layers in vertical direction and assign each device with a subset of these neurons to execute calculations. As this method splits the neural network this method can be possibly applied to deploy large neural networks which normally can not be deployed in a single device. Furthermore, model parallelism is used for training large neural networks in GPU clusters. Therefore applying this method for the inference task is sound feasible and promising.

2.2 Related Work

Improving Deep Learning inference performance to overcome challenges it has faced is an intensively researched area [1]. Inference is computationally costly when it comes to high dimensional inputs. Therefore, a lot of calculations have to be done on that input data. One common solution was to use cloud computing as a solution, as they can use cloud resources for computation. But moving the data from the source to cloud introduced some challenges such as latency, scalability and privacy [1]. When it comes to real time applications, latency incurred by additional queuing and propagation delays of the network is not acceptable. Unpredictability of the remote cloud server status is also a problem. Uploading all the data from the source to the cloud is inefficient too. Sometimes there may be privacy issues when uploading data. Therefore, to avoid those challenges edge computing came into the play. Edge computing reduced the latency by doing computations in close proximity to the end devices. A hierarchical structure and a local edge server can be used to avoid privacy issues. But edge computing also introduced new challenges. One of the main challenges is using the less powerful edge computing devices for the high resource requirements in deep learning applications. An edge device may be

an edge server with a GPU or a mobile device or it may be even a Raspberry Pi device. Another challenge was to coordinate with edge devices to produce an output.

As mentioned in [1], there are three approaches for Deep learning inference in edge computing scenarios.

1. All computations done on the end device.
2. Edge server computation.
3. Computation across edge devices.

In the first method, the model is located at the edge device. As the edge devices are not capable of doing heavy computation and memory intensive tasks, those models need to be designed with less parameters. Otherwise, later the parameters should be compressed or unnecessary parameters should be removed. In DeepIoT [2], the authors present a pruning method for DNN models used in IoT devices. But this may have an impact on the results by reducing the model accuracy. Another sub method of this is where application-specific hardware is used for solutions in order to speed up the process and save energy [1].

Though all those methods are considered, the first method is still challenging because of the resource limitations such as power, computation and memory [1]. Because of that some researchers tend to use the second method which involves an edge server. As done in the deep learning framework presented by the authors of paper [3], where they offload the whole DNN to the edge server for computations. But sometimes it is helpful to do data preprocessing to reduce the communication time with the edge server. In [4], they use removing blurry images, and cropping to take only the required parts of images as preprocessing techniques. A main problem faced here is resource management in the edge server as multiple end devices may be using the same shared edge server. Because of that tradeoff between latency, accuracy and other performance metrics need to be considered. Some solutions such as Mainstream [5] use transfer learning, where common lower layers of the DNN model are used for multiple applications to achieve the above mentioned tradeoff.

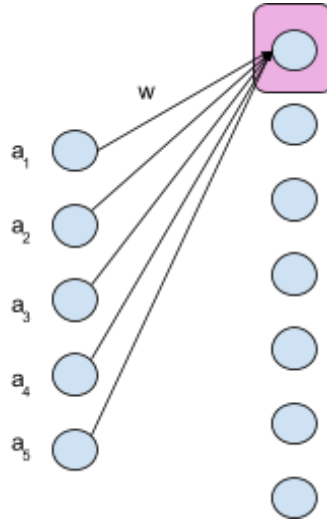
Although the second method increases the speed of inference, there is a third method more efficient than the above method, where intelligent offloading is considered. Solutions done in this method face challenges such as taking a decision whether to offload the entire DNN or not (depends on size of the data, hardware abilities, DNN model, network quality), and which fractions of a DNN computations should be offloaded [1]. When using DNN partitioning, some solutions have used layer-wise partitioning as in Neurosurgeon [6] and some have used input-wise partitioning as in MoDNN [7] and DeepThings [8]. Input-wise partitioning also has a problem of increased data dependency. Some solutions such as DDNN [9] are using a hierarchy of cloud, edge servers and end devices to model partitioning. Cloud is used only if total processing time is reduced, because of the available powerful resources and increased privacy risk.

Based on our literature survey, we saw a lack of research done in the area of distributed deep learning inference across multiple helper edge devices without using a remote cloud. Latency and task management are the main issues which are not much addressed in the existing solutions. Latency is caused by less efficient load balancing heuristics and dynamic task sharing systems which we will be improving in our solution. Using fine-grained partitioning of the DNN executions across edge devices, considering their resource availability and network condition seems like a promising method for our solution according to the research.

3. Methodology

Based on the literature review, we identified that offloading deep learning inference computation across edge devices as a suitable approach. We first describe how we plan to partition the inferencing tasks based on model parallelism. Then, the planned task sharing mechanism among edge devices will be described.

3.1 Model Parallelism Based Partitioning

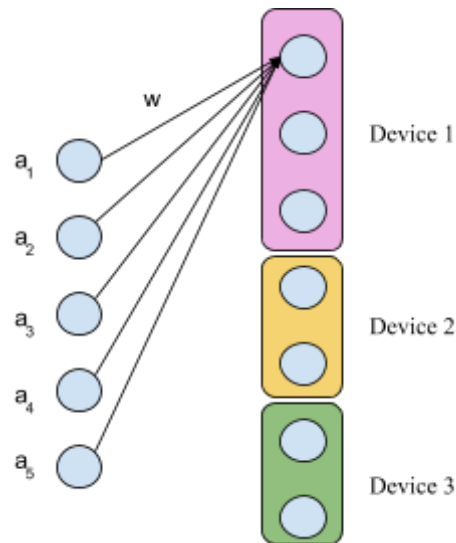


As shown in the above figure the partitioning can be obtained by the output partitioning of the network. By this method the above two layers can be partitioned into seven independently executable tasks. Each task will be added to a task queue in the main device. Also by partitioning this way we can ensure that even a single device can work without any worker devices.

This partitioning enables the ability to distribute the workload among a varying number of worker devices depending on their availability. There will be two configuration options for the distribution method.

1. When the number of worker devices in the network is fixed.
2. When the number of worker devices in the network is dynamic.

If the number of worker devices are fixed and known, the main device will distribute tasks evenly among all devices as shown in the below figure.



If the number of worker devices are dynamic, the main device will only distribute tasks in the task queue when a task sharing request is received. Otherwise, it will continue on executing all remaining tasks in the queue.

3.2 Communication and Task sharing

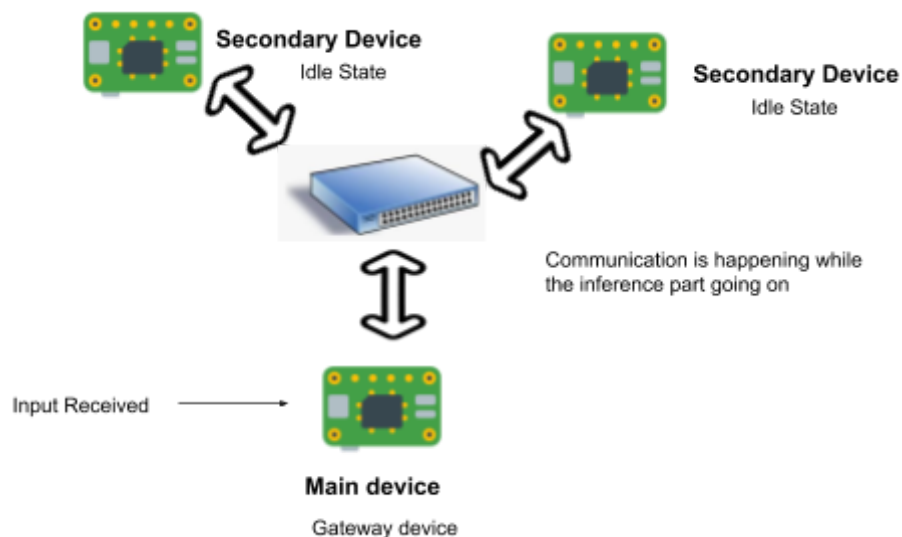


Figure 4. Raspberry pi Communication cluster

As shown in the above figure (figure 4) the main device (the gateway device) will receive input data through its sensors. In this case we assume that there are two other devices on the same network which are in idle state or doing less computationally expensive tasks. These secondary devices also can be changed to a main device (the gateway device) dynamically when it receives an input. All three devices have trained Image Classification model and the partitioning and sharing services running in the background.

The main device which captured the image will run the partitioning program to partition the neural network and input data to separately executable tasks and it will add them to a task queue. Then the main device will start the inference execution by taking one task by one from the queue. Meanwhile, it will broadcast a task sharing request among the network and secondary devices which are in idle state will capture this request and they will notify the main device that they are available to participate. The main device will then start sending a set of tasks from the queue to secondary devices. When the secondary devices have finished the calculation in each neuron they will update the main device with results and request more tasks from the queue until it is empty or the execution is interrupted due to its own requirements of executing its own process.

We are planning to implement the proposed system using C++ programming language as the C++ language is more compatible with low level devices like Raspberry Pis and it helps to gain faster execution time.

3.3 Research Timeline

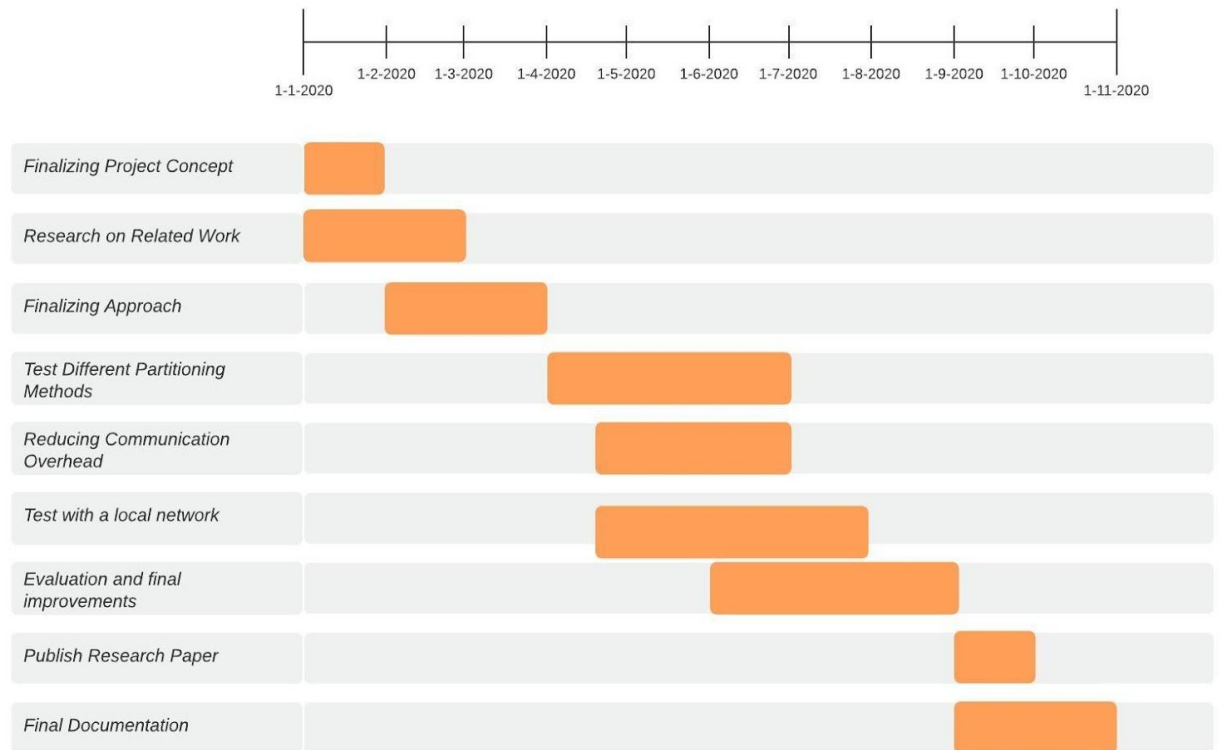


Figure 5. Tentative research timeline

4. Conclusion/Summary

In conclusion, this project is intended to propose and develop a novel approach that can be used to distributively execute the inference of fully connected neural networks. We hope that this distributed Raspberry Pi based partitioning method will reduce the time taken for inference tasks and make it possible to run large neural networks which usually cannot be deployed to a single device. Furthermore, we plan to investigate the ideal method for partitioning to reduce communication demand among these Raspberry Pis to prevent unwanted communication overheads. We hope this research will help to boost future IoT based and edge computing applications capabilities.

References

- [1] J. Chen and X. Ran, “Deep Learning With Edge Computing: A Review,” *Proc. IEEE*, vol. 107, no. 8, 2019.
- [2] S. Liu, Y. Lin, Z. Zhou, K. Nan, H. Liu, and J. Du, “DeepIoT: Compressing deep neural network structures for sensing systems with a compressor-critic framework,” in *Proc. SenSys*, 2017, pp. 1–4.
- [3] X. Wang, X. Wang, and S. Mao, “RF sensing in the Internet of Things: A general deep learning framework,” *IEEE Commun. Mag.*, vol. 56, no. 9, pp. 62–67, Sep. 2018.
- [4] C. Liu et al., “A new deep learning-based food recognition system for dietary assessment on an edge computing service infrastructure,” *IEEE Trans. Services Comput.*, vol. 11, no. 2, pp. 249–261, Jan. 2018.
- [5] A. H. Jiang et al., “Mainstream: Dynamic stream-sharing for multi-tenant video processing,” in *Proc. USENIX Annu. Tech. Conf. (USENIXATC)*, 2018, pp. 29–42.
- [6] Y. Kang et al., “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 615–629, 2017.
- [7] J. Mao, X. Chen, K. W. Nixon, C. Krieger, and Y. Chen, “MoDNN: Local distributed mobile computing system for deep neural network,” in *Proc. Design, Autom. Test Eur. Conf. Exhibit. (DATE)*, Mar. 2017, pp. 1396–1401.
- [8] Z. Zhao, K. M. Barijough, and A. Gerstlauer, “Deepthings: Distributed adaptive deep learning inference on resource-constrained iot edge clusters,” *IEEE Trans.*

Comput.-Aided Design Integr. Circuits Syst., vol. 37, no. 11, pp. 2348–2359, Nov. 2018.

- [9] S. Teerapittayanon, B. McDanel, and H. Kung, “Distributed deep neural networks over the cloud, the edge and end devices,” in Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS), Jun. 2017, pp. 328–339.
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” IEEE Internet of Things Journal (IoT-J), vol. 3, no. 5, pp. 637–646, 2016.
- [11] Leigh Johnson “Portable Computer Vision: TensorFlow 2.0 on a Raspberry Pi” online: <https://towardsdatascience.com/portable-computer-vision-tensorflow-2-0-on-a-raspberry-pi-part-1-of-2-84e318798ce9>.
- [12] R. Stahl, Z. Zhao, D. Mueller-Gritschneider, A. Gerstlauer, and U. Schlichtmann, “Fully distributed deep learning inference on resource-constrained edge devices,” in International Conference on Embedded Computer Systems . Springer, 2019, pp. 77–90.
- [13] F. Iandola, “Exploring the Design Space of Deep Convolutional Neural Networks at Large Scale,” no. December 2016, 2016.